gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

# DAT321 DIT843
# Specification-based Testing

Dr. Francisco Gomes de Oliveira Neto (he/him)

## Myriung Kim

- Professor at the University of California (UCLA)
- Major contributions on automated testing, debugging and refactoring
- 2022 ACM SIGSOFT Influential Educator Award

# What we will cover today?

- What is **specification-based testing**?
- What are **Equivalent (Classes) Partitions (EP)** in testing?
- How can we use partitions to **design test cases**?
- What is **Diversity**-based Testing?
- How can we use diversity with EP?


© Francisco Oliveira Neto - 2018

# Part 1:

# Specification and Equivalence Partitioning

# Software Testing and the Specification

- Software Testing fulfils **two complementary purposes**:
  1. **Reveal failures** that can unearth faults.
  2. Help us **validate our understanding** of the specification.

     Train our “gut feeling” as engineers.

- The second point above is **particularly important** if the specification is not written down or not well defined.

© Francisco Oliveira Neto - 2018

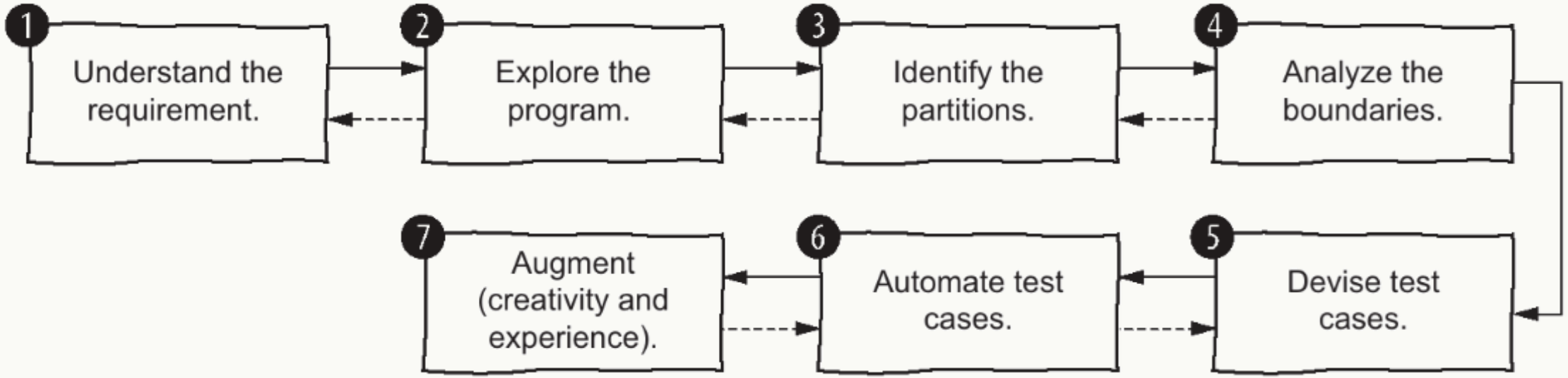
# Specification-based Testing - What

- Also referred to as **black-box testing**.
- In essence, spec-based rely on information from the **input space** (i.e., possible values for input) and the expected output.

- **Depends on** the clarity of **specification**, which can vary.
  - Can be precise and written, but can also be abstract and unclear.
  - Spec-based testing is better when the specification is more clear.
- Black-box can be done in **all levels of testing**.


© Francisco Oliveira Neto - 2018

# Specification-based Testing - How





© Francisco Oliveira Neto - 2018

# The Input Space / Input Domain

- The **input space** is the set of all input values that can be given to the program.
- For even small programs, the **input can be infinite**. Therefore, designing tests is basically choosing fininte sets in the input space.

- The space can be **multidimensional** (see Exercises 2 and 3).


© Francisco Oliveira Neto - 2018

# What is Equivalence Partitioning?

- EP is a technique to identify **sets of test input** that trigger the **same SUT behaviour**.
  - **Important:** Same behaviour does not mean literary the same output.
  - Examples: Sets of input that throw exception with different messages can still belong to the same partition. Focus on different **assumptions related to behaviour**.

- EP requires **creativity + knowledge** of the specification.

© Francisco Oliveira Neto - 2018

# How to partition classes?

1. Identify **how many parameters**, and the types of those parameters.
2. **For each** parameter, **extract properties** related to the type.
   - Properties is any unique characteristics that can “group” input values.
   - Most properties are domain-specific, but some are typical
     - **String** properties: length, blank spaces, null string, number of special chars
     - **Numerical** properties: positive, negative, zero, max and min values for a type
     - **Data structures**: empty or null, sorted, has the same elements, number of elements, types of the elements (Polymorphism).
3. Try to understand the expected output created by those inputs. They should convey the same semantics about the program (e.g., cover the same ‘case’ or ‘requirement’)


© Francisco Oliveira Neto - 2018

# Exercise 1 – Grading Courses

- Let us test a method that receives the exam score of a student (integer) and then outputs the grade as a String. Values below zero and above 100 should throw different exceptions.
  - **Input:** Integer value.
  - **Output:** A string for the course grade.

```
Grade:
Exception: score < 0
U:  0 <= score < 50
3: 50 <= score < 70
4: 70 <= score < 85
5: 85 <= score <= 100
Exception: score > 100
```

- Examples:
  - `in= 100, out="5"; in = -5, out = Exception("Cannot be negative").`
  - `in = 232534, out = Exception("Must be between 0 and 100").`
  - `in = 10, out = "U", in = 50, out = "3".`
- Partitions from ChatGPT (Note: they are incomplete and redundant!)


© Francisco Oliveira Neto - 2018

# Exercise 2 – Array input

- Let us test a simple method that **returns the index of the highest number** in an array. Specification:
  - **Input:** Array of integers
  - **Output:** The index of the highest number in the vector.
  - In case the arrays is empty, return the index -1.

- Example:
  - `In: [], Out:-1; In: [1, 2, 3], Out: 2;`


© Francisco Oliveira Neto - 2018

# Part 2:

# Diversity-based Testing

# Example Choosing Tests – Phonebook app

| TC1 - Add contact (create) |
| --- |
| Select “Add contact” |
| Choose “Create New” |
| Fill in fields |
| Press ”Include contact” |

| TC2 – Add contact (import) |
| --- |
| Select “Add contact” |
| Choose “Import” |
| Select an existing contact |
| Press ”Include contact” |

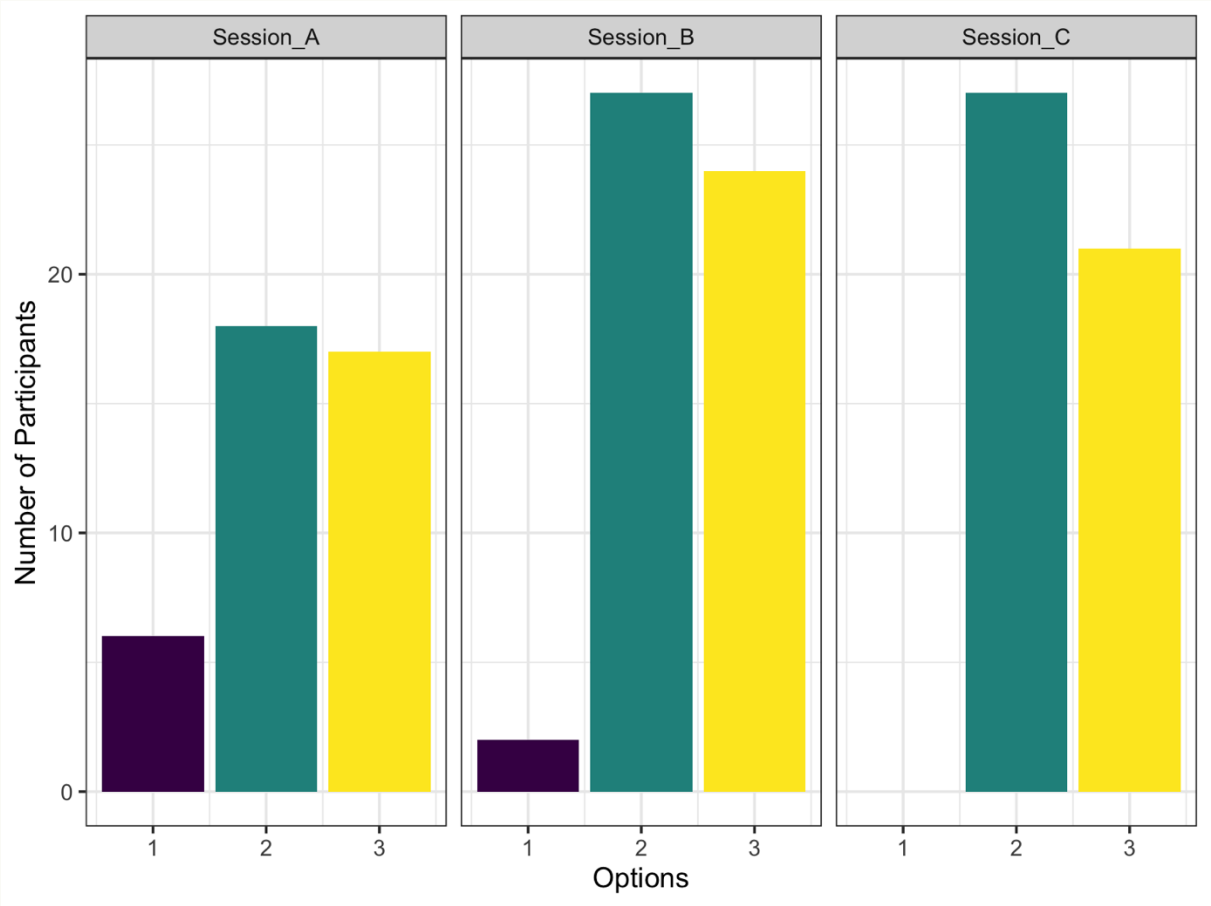
| TC3 – Remove a contact |
| --- |
| Select “Remove contact” |
| Select one contact |
| Press “Remove Contact(s)” |

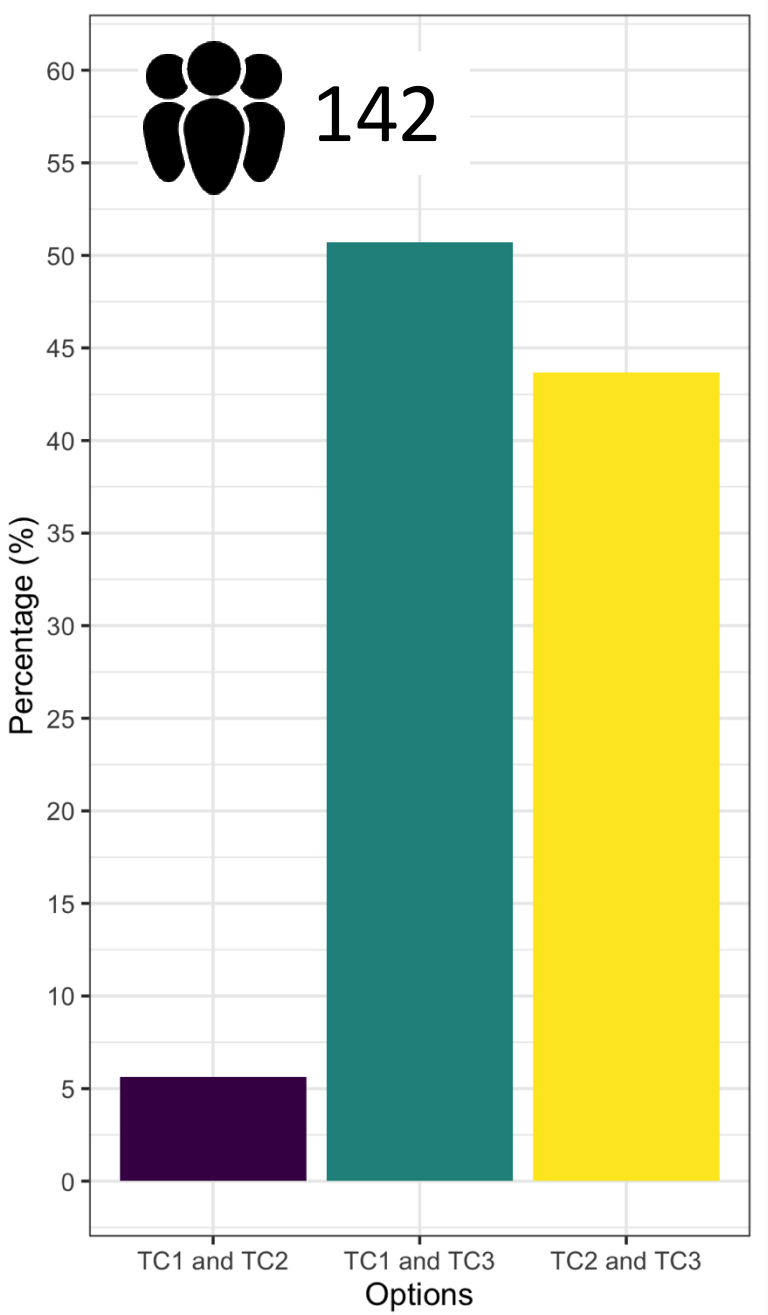
1. TC1 and TC2
2. TC1 and TC3
3. TC2 and TC3

© Francisco Oliveira Neto - 2018

# Data from previous presentations

41 53 48



# The real question…

~~Which~~ pair??

**Why that**


© Francisco Oliveira Neto - 2018

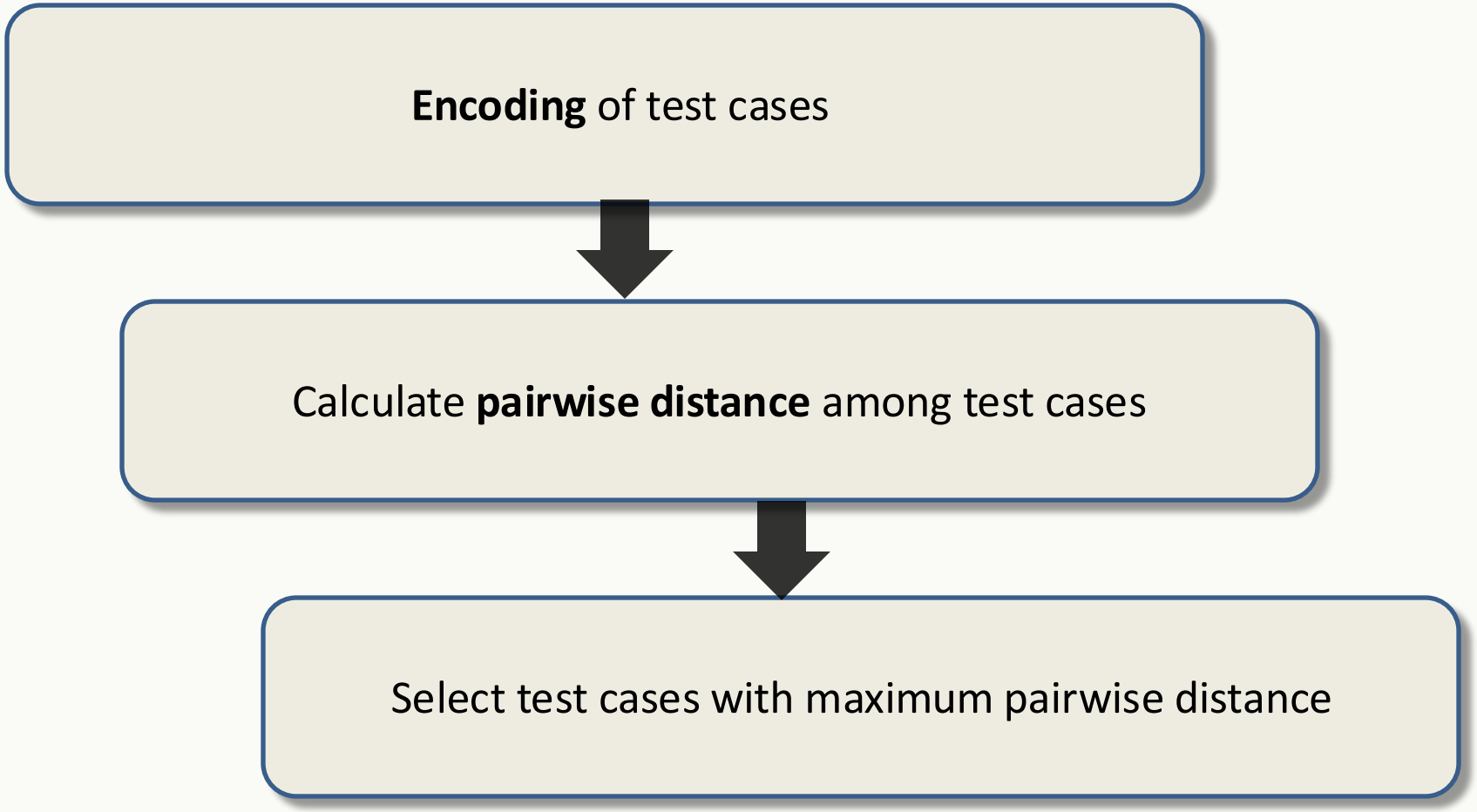
# When choosing tests…

- We tend to be **diverse**…
- Specially if we **don't know** what to test.

**Similar tests,** tend to reveal the **same defect**.


© Francisco Oliveira Neto - 2018

# Diversity-based Testing (DBT)




Adapted from Hadi Hemmati, Andrea Arcuri, and Lionel Briand. 2013. Achieving scalable model-based testing through test case diversity. *ACM Trans. Softw. Eng. Methodol.* 22, 1, Article 6 (March 2013), 42 pages. DOI=http://dx.doi.org/10.1145/2430536.2430540

# Encoding: Diversity is a number

| TC1 - Add contact (create) |
|---|
| Select “Add contact” |
| Choose “Create New” |
| Fill in fields |
| Press ”Include contact” |

| TC2 – Add contact (import) |
|---|
| Select “Add contact” |
| Choose “Import” |
| Select an existing contact |
| Press ”Include contact” |

| TC3 – Remove a contact |
|---|
| Select “Remove contact” |
| Select one contact |
| Press “Remove Contact(s)” |

© Francisco Oliveira Neto - 2018

# Encoding: Distance / Similarity functions

| | |
|---|---|
| $NCD_Z(x,y) = \frac{Z(xy) - \min\{Z(x), Z(y)\}}{\max\{Z(x), Z(y)\}}.$ | **Normalized Compressed Distance** |
| $d(\vec{x},\vec{y}) = \sqrt{(\vec{x}-\vec{y})^T S^{-1} (\vec{x}-\vec{y})}.$ | **Mahalanobis** |
| $\mathrm{NGD}(x,y) = \frac{\max\{\log f(x), \log f(y)\} - \log f(x,y)}{\log N - \min\{\log f(x), \log f(y)\}}$ | **Normalized Google Distance** |
| $d_J(A,B) = 1 - J(A,B) = \frac{\lvert A \cup B\rvert - \lvert A \cap B\rvert}{\lvert A \cup B\rvert}.$ | **Jaccard Distance** |
| $\mathrm{lev}_{a,b}(i,j) = \begin{cases} \max(i,j) \\ \min \begin{cases} \mathrm{lev}_{a,b}(i-1,j) + 1 \\ \mathrm{lev}_{a,b}(i,j-1) + 1 \\ \mathrm{lev}_{a,b}(i-1,j-1) + 1_{(a_i \neq b_j)} \end{cases} \end{cases}$ | **Lenvenshtein Distance** |

# Diversity-based Testing - Example

| TC1 - New contact (create) |
|---|
| Select “Add contact” |
| Choose “Create New” |
| Fill in fields |
| Press ”Include contact” |

{ add, choose, contact, create, fields, fill, in, include, new, press, select }

| TC2 – New contact (import) |
|---|
| Select “Add contact” |
| Choose “Import” |
| Select an existing contact |
| Press ”Include contact” |

{ add, an, choose, contact, existing, import, include, press, select }

| TC3 – Remove a contact |
|---|
| Select “Remove contact” |
| Select one contact |
| Press “Remove Contact(s)” |

{ contact, contact(s), one, press, remove, select }

# Diversity-based Testing - Example

| TC1 - New contact (create) |
| --- |
| Select “Add contact” |
| Choose “Create New” |
| Fill in fields |
| Press ”Include contact” |

| TC2 – New contact (import) |
| --- |
| Select “Add contact” |
| Choose “Import” |
| Select an existing contact |
| Press ”Include contact” |

| TC3 – Remove one contact |
| --- |
| Select “Remove contact” |
| Select one contact |
| Press “Remove Contact(s)” |

| TC4 – Remove several contacts |
| --- |
| Select “Remove contact” |
| Select several contacts |
| Press “Remove Contact(s)” |

| TC5 – Update contact |
| --- |
| Select “Update contact” |
| Choose one contact and field |
| Update the chosen field |
| Press “Confirm update” |

# Diversity-based Testing - Example

TC1 = {add, choose, contact, create, fields, fill, in, include, new, press, select}

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC5 = {and, choose, chosen, confirm, contact, field, one, press, select, the, update}

# Diversity-based Testing - Example

| | TC1 | TC2 | TC3 | TC4 | TC5 |
|---|---|---|---|---|---|
| TC1 | | | | | |
| TC2 | | | | | |
| TC3 | | | | | |
| TC4 | | | | | |
| TC5 | | | | | |

We now need all **pairwise distance** / similarity values. Typically, we arrange them in a **matrix**. **Each test** case is a **row** and a **column** in the matrix.

# Diversity-based Testing - Example

**TC2 =** {add, an, choose, contact, existing, import, include, press, select}

**TC4 =** {contact, contact(s), contacts, press, remove, select, several}

$$J(TC_i, TC_j) = \frac{|TC_i \cap TC_j|}{|TC_i \cup TC_j|}$$

# Diversity-based Testing - Example

**TC2 =** {add, an, choose, contact, existing, import, include, press, select}

**TC4 =** {contact, contact(s), contacts, press, remove, select, several}

**TC2 ∪ TC4 =** {add, include, select, import, several, choose, an, contact(s), remove, existing, contact, press, contacts}

**TC2 ∩ TC4 =** {select, contact, press}

**| TC2 | = 9** **| TC2 ∪ TC4 | = 13**

**| TC4 | = 7** **| TC2 ∩ TC4 | = 3**

$$J(TC_2, TC_4) = \frac{|TC_3 \cap TC_4|}{|TC_3 \cup TC_4|} = \frac{3}{13} = 0.23$$

$$Distance(TC2, TC4) = 1 - 0.23 = 0.77$$

**Note:** Jaccard Index uses a **similarity index**, instead of a diversity index. So, **higher** values means **more similar** test cases. And for diversity, **we want low similarity** values.

# Diversity-based Testing - Example

**TC3 =** {contact, contact(s), one, press, remove, select}

**TC4 =** {contact, contact(s), contacts, press, remove, select, several}

**TC3 ∪ TC4 =** {select, contact, one, several, contact(s), press, remove, contacts}

**TC3 ∩ TC4 =** {select, contact, contact(s), press, remove}

**| TC3 | = 6** **| TC3 ∪ TC4 | = 8**

**| TC4 | = 7** **| TC3 ∩ TC4 | = 5**

$$J(TC_3, TC_4) = \frac{|TC_3 \cap TC_4|}{|TC_3 \cup TC_4|} = \frac{5}{8} = 0.63$$

$$Distance(TC_3, TC_4) = 1 - 0.63 = 0.37$$

# Diversity-based Testing - Example

| | TC1 | TC2 | TC3 | TC4 | TC5 |
|---|---|---|---|---|---|
| TC1 | | | | | |
| TC2 | | | | 0.77 | |
| TC3 | | | | 0.37 | |
| TC4 | | 0.77 | 0.37 | | |
| TC5 | | | | | |

$$D(TC_2, TC_4) = D(TC_4, TC_2) = 0.77$$

$$D(TC_3, TC_4) = D(TC_3, TC_4) = 0.37$$

# Diversity-based Testing - Example

| | TC1 | TC2 | TC3 | TC4 | TC5 |
|---|---|---|---|---|---|
| TC1 | | | | | |
| TC2 | | | | 0.77 | |
| TC3 | | | | 0.37 | |
| TC4 | | 0.77 | 0.37 | | |
| TC5 | | | | | |

$$D(TC_2, TC_4) = D(TC_4, TC_2) = 0.77$$

$$D(TC_3, TC_4) = D(TC_3, TC_4) = 0.37$$

- Notice that 3 and 4 are more similar (i.e., "closer" to each other) if compared with TC2 and TC4.
- Using **distance values** we can find the **more "different"** tests from each other.

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

# Exercise

- Calculate the rest of the similarity values.

TC1 = {add, choose, contact, create, fields, fill, in, include, new, press, select}

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC5 = {and, choose, chosen, confirm, contact, field, one, press, select, the, update}

# Exercise

- Calculate the rest of the similarity values.

TC1 = {add, choose, contact, create, fields, fill, in, include, new, press, select}

TC2 = {add, an, choose, contact, existing, import, include, press, select}

TC3 = {contact, contact(s), one, press, remove, select}

TC4 = {contact, contact(s), contacts, press, remove, select, several}

TC5 = {and, choose, chosen, confirm, contact, field, one, press, select, the, update}

# Diversity-based Testing - Example

| | TC1 | TC2 | TC3 | TC4 | TC5 |
|---|---|---|---|---|---|
| TC1 | --- | 0.57 | 0.78 | 0.80 | 0.78 |
| TC2 | 0.57 | --- | 0.75 | 0.77 | 0.75 |
| TC3 | 0.78 | 0.75 | --- | 0.37 | 0.69 |
| TC4 | 0.80 | 0.77 | 0.37 | --- | 0.80 |
| TC5 | 0.78 | 0.75 | 0.69 | 0.80 | --- |

This is a similarity / distance matrix.

The **matrix is symetrical**, because:

D(a,b) == D(b,a)

The **diagonal is ignored**, because the distance of an object to itself is always zero (or similarity is always one).

# Diversity-based Testing - Example

- We can use the matrix for all sorts of operations.
- We can find diverse subsets: Pick the **highest values one at a time**.
- We can see whether we need to create more different tests.

| | TC1 | TC2 | TC3 | TC4 | TC5 |
|---|---|---|---|---|---|
| TC1 | --- | 0.57 | 0.78 | 0.80 | 0.78 |
| TC2 | | --- | 0.75 | 0.77 | 0.75 |
| TC3 | | | --- | 0.37 | 0.69 |
| TC4 | | | | --- | 0.80 |
| TC5 | | | | | --- |